17. Using JPA Entities for Database Persistence

This video, I'm going to go ahead and start creating our persistence layer. What we are going to be doing is we're going to be using JPA, and in this section of the course, I am going to go through it a little fast, and we will be coming back and going into things a lot more detail. So I know you're going to have questions about what we need to be creating and how things are being created, but I do want to like go through this, create a nice little web application so you can see how quickly you can build something using Spring Boot and JPA. So I'm going to come in here, I'm going to create a new package. I'll call that domain. And people keep their JPA entities in several different packages. Domain is fairly common. And what we're going to do is we'll set up a traditional book author setup, we'll do author, and I'll be adding this. We don't need to keep popping that question up. And then we will also add in a book. So I'm sure in some programming examples of object oriented programming, we've probably seen this example before. So let's work on author first here. We'll do private string first name like so. Author is just going to have a simple first name last name. And for the book. So we'll do title for the book. And most books have an has been and we'll make a string property ISBN. So at this point this has set up our basic data structure. Right now we just have a couple of pojos. I haven't had it in the getters and setters yet. I'll do that in a moment. But just to give you an idea of what we are building here. So an author and a book relationship. And for JPA mapping I'll take care of author first. This is also going to have a property an ID value. So this ID value is what's going to be used to persist it into the database. So let's come in here. And if we use Jakarta persistence. This is what's telling JPA that this is going to be an entity persisted to the database. So now I have a red line there because I have not given an ID value. So I'm going to come in here say id and then this annotation here I'm going to say generated value. So we're going to say that the ID gets generated. It's going to be by the database. So here I need to say strategy generation type auto. And I need to do the exact same thing in book. So we want to annotate this. As an entity we want to give it an ID value. And then we are also going to annotate this as an ID. Again you want to be sure that you're using Jakarta Persistence and not the spring data annotation for ID. Definitely use persistence one that is the JPA specification. And then we also want to say generated

value. Again here we're saying a strategy. So we're saying that the database is going to provide that. So here we'll be using H2 which does have an auto increment column. So at this point we have two entities created. We have an author entity and a book entity. So we just have private values on them. We do want to come in and generate getters and setters. So I just did command n that's on a mac keyboard to bring up getters and setters. So I'm just going to tell IntelliJ to do that. And we'll generate getters and setters for all of these. So they are private values. So we do need getters and setters for each. And let's come into book also do the same thing again. Getters and setters. And we'll go ahead and highlight those and say okay. So what we have at this point is two Pojos Pojo being plain old Java object. And we are providing getters and setters for all the private properties. So very simple implementation at this point. Right now we do not have a relationship between author and book, and I will complete that in the next video.

```
spring-7-webapp/
 ├── .mvn
 ├── src/
 |    ├── main/
 |    |    ├── java/
 |    |    |    └── guru.springframework.spring6webapp/
 |    |    |        ├── domain/
 |    |    |        |   └── domain/
 |    |    |        |       ├── Author.java # class
 |    |    |        |       └── Book.java   # class
 |    |    |        └── Spring6WebappApplication.java
 |    |    └── resources/
 |    |         ├── static/
 |    |         ├── templates/
 |    |         └── application.yaml
 |    └── test/
 |         ├── java/
 |         |    └── guru.springframework.spring6webapp/
 |         |        └── Spring6WebappApplicationTests.java
 |         └── resources/
 ├── .gitattributes
 ├── .gitignore
 ├── HELP.md
```

```
  ├── mvnw
  ├── mvnw.cmd
  ├── pom.xml
  └── README.md
```

```
@Entity
public class Author {

  @Id
  @GeneratedValue(strategy = GenerationType.AUTO)
  private Long id;
  private String firstName;
  private String lastName;

  // getter and setter

}

@Entity
public class Book {

  @Id
  @GeneratedValue(strategy = GenerationType.AUTO)
  private Long id;
  private String title;
  private String isbn;

  // getter and setter

}
```

Utilizando entidades JPA para persistência de banco de dados

Neste vídeo, vou começar a criar nossa **camada de persistência**. O que faremos é utilizar **JPA** e, nesta parte do curso, vou avançar um pouco mais

rápido. Mais adiante, retornaremos a esses conceitos e entraremos em muito mais detalhes.

Eu sei que você provavelmente terá dúvidas sobre o que precisamos criar e sobre como as coisas estão sendo criadas, mas a minha intenção aqui é percorrer esse processo, construir uma aplicação web simples e agradável, para que você possa ver o quão rapidamente é possível desenvolver algo utilizando **Spring Boot** e **JPA**.

Vou acessar o projeto e criar um novo pacote, que chamarei de `domain`. As pessoas organizam suas entidades JPA em diferentes pacotes, e `domain` é uma convenção bastante comum.

O que vamos fazer é configurar um modelo tradicional de **livros e autores**. Começaremos com `Author`. Não precisamos continuar exibindo aquela pergunta de confirmação. Em seguida, também adicionaremos a entidade `Book`.

Esse é um exemplo bastante comum em programação orientada a objetos, e você provavelmente já o viu antes em outros contextos. Vamos começar trabalhando com a classe `Author`.

Definiremos um atributo privado `String firstName`, dessa forma. A classe `Author` terá apenas informações simples: **nome** e **sobrenome**.

Para a entidade `Book`, teremos o atributo `title`. Além disso, a maioria dos livros possui um ISBN, então criaremos uma propriedade do tipo `String` chamada `isbn`.

Nesse ponto, já temos definida nossa **estrutura básica de dados**. Atualmente, temos apenas alguns **POJOs**. Ainda não adicionei os *getters* e *setters*, mas farei isso em instantes. A ideia aqui é apenas mostrar o que estamos construindo: um relacionamento entre **autor** e **livro**.

Para o mapeamento com JPA, começarei pela entidade `Author`. Ela também terá um atributo `id`. Esse valor de ID será utilizado para persistir a entidade no banco de dados.

Vamos então utilizar as anotações de **Jakarta Persistence**. Essa anotação informa ao JPA que essa classe será uma entidade persistida no banco de dados. Neste momento, o IDE exibe um erro, pois ainda não definimos um identificador (`ID`).

Vamos então adicionar o atributo `id` e utilizar a anotação `@GeneratedValue`. Com isso, indicamos que o valor do ID será gerado automaticamente pelo banco de dados. Aqui, definimos a estratégia como `GenerationType.AUTO`.

Precisamos fazer exatamente o mesmo na entidade `Book`. Devemos anotá-la como uma entidade, definir um identificador e marcar esse atributo como `@Id`.

É muito importante garantir que você esteja utilizando as anotações do **Jakarta Persistence**, e não as anotações de ID do Spring Data. O correto é usar as anotações da especificação JPA.

Também utilizaremos `@GeneratedValue` com a estratégia automática, indicando que o banco de dados será responsável por fornecer esse valor. Neste projeto, utilizaremos o banco **H2**, que possui suporte a colunas com *auto increment*.

Neste ponto, temos duas entidades criadas: `Author` e `Book`. Ambas possuem apenas atributos privados. Agora precisamos gerar os *getters* e *setters*.

No meu caso, utilizei o atalho `Command + N` no teclado do Mac para gerar automaticamente os *getters* e *setters* no IntelliJ IDEA. Vou gerar esses métodos para todos os atributos, já que são privados e precisam ser acessados.

Em seguida, faremos o mesmo para a classe `Book`, gerando novamente todos os *getters* e *setters*.

Agora temos dois **POJOs** (*Plain Old Java Objects*), cada um com seus respectivos métodos de acesso para todas as propriedades privadas. Trata-se de uma implementação bastante simples neste momento.

Por enquanto, **ainda não existe um relacionamento entre `Author` e `Book`**. Esse relacionamento será implementado no próximo vídeo.

Estrutura do Projeto

```
spring-7-webapp/
 ├── .mvn
 ├── src/
 |    ├── main/
 |    |    ├── java/
 |    |    |    └── guru.springframework.spring6webapp/
 |    |    |        ├── domain/
 |    |    |        |   └── domain/
 |    |    |        |       ├── Author.java # class
 |    |    |        |       └── Book.java   # class
 |    |    |        └── Spring6WebappApplication.java
 |    |    └── resources/
 |    |         ├── static/
```

```
|    |         ├── templates/
|    |         └── application.yaml
|    └── test/
|         ├── java/
|         |    └── guru.springframework.spring6webapp/
|         |        └── Spring6WebappApplicationTests.java
|         └── resources/
├── .gitattributes
├── .gitignore
├── HELP.md
├── mvnw
├── mvnw.cmd
├── pom.xml
└── README.md
```

Código das Entidades

```
@Entity
public class Author {

  @Id
  @GeneratedValue(strategy = GenerationType.AUTO)
  private Long id;
  private String firstName;
  private String lastName;

  // getter and setter

}
```

```
@Entity
public class Book {

  @Id
  @GeneratedValue(strategy = GenerationType.AUTO)
  private Long id;
  private String title;
```

```
  private String isbn;

  // getter and setter

}
```